The Docxology Template: A Modular Ecosystem for High-Integrity Research

Architecture, Ergonomics, and Automated Provenance in Computational Science

Daniel Ari Friedman

Active Inference Institute

daniel@activeinference.institute

ORCID: 0000-0001-6232-9096

DOI: 10.5281/zenodo.template.meta

2026-03-09

2026-03-09

Contents

1 Abstract **4**

2 Introduction **5**
- 2.1 The Reproducibility Crisis and Its Structural Roots . . . 5
- 2.2 Related Work . . . 5
  - 2.2.1 Workflow Managers . . . 5
  - 2.2.2 Literate Programming and Publication Tools . . . 5
  - 2.2.3 Containerization and Environment Capture . . . 6
  - 2.2.4 Best-Practice Frameworks and Data Standards . . . 6
  - 2.2.5 The Gap . . . 6
- 2.3 The Docxology Template: An Integrated Solution . . . 6
- 2.4 Scope and Contributions . . . 7
- 2.5 Paper Organization . . . 7

3 Methods **8**
- 3.1 The Two-Layer Architecture . . . 8
- 3.2 The Standalone Project Paradigm . . . 8
- 3.3 The Thin Orchestrator Pattern . . . 8
- 3.4 The Eight-Stage Pipeline . . . 9
  - 3.4.1 Stage 00: Environment Setup (`00_setup_environment.py`) . . . 9
  - 3.4.2 Stage 01: Test Execution (`01_run_tests.py`) . . . 9
  - 3.4.3 Stage 02: Analysis Execution (`02_run_analysis.py`) . . . 9
  - 3.4.4 Stage 03: PDF Rendering (`03_render_pdf.py`) . . . 9
  - 3.4.5 Stage 04: Output Validation (`04_validate_output.py`) . . . 9
  - 3.4.6 Stage 05: Output Organization (`05_copy_outputs.py`) . . . 10
  - 3.4.7 Stage 06: LLM Review (`06_llm_review.py`) . . . 10
  - 3.4.8 Stage 07: Executive Report (`07_generate_executive_report.py`) . . . 10
- 3.5 The Interactive Orchestrators . . . 10
  - 3.5.1 `run.sh`: The Standard Orchestrator . . . 10
  - 3.5.2 `secure_run.sh`: The Steganographic Superset . . . 10
- 3.6 Documentation Duality and AI Collaboration . . . 10
  - 3.6.1 FAIR Alignment . . . 11

3.7 Quality Assurance . . . 11
3.7.1 Zero-Mock Testing Policy . . . 11
3.7.2 Visualization Standards . . . 11

4 Results **12**
4.1 Multi-Project Pipeline Execution . . . 12
4.2 Infrastructure Test Suite . . . 12
4.3 Infrastructure Module Inventory . . . 12
4.4 Pipeline Stage Execution . . . 13
4.5 Steganographic Performance . . . 13
4.6 Self-Referential Analysis . . . 14
4.7 Comparative Feature Analysis . . . 15
4.8 Test Quality Metrics . . . 16

5 Discussion **17**
5.1 The Zero-Mock Tradeoff . . . 17
5.2 Scalability: From 1 to N Projects . . . 17
5.3 Comparison to Existing Tools . . . 17
5.4 The AI Collaboration Model . . . 18
5.5 The Learning Curve . . . 18
5.6 Limitations . . . 19
5.7 Future Directions . . . 19

6 Conclusion **19**

7 Infrastructure Module Reference **21**
7.1 `infrastructure.core` (28 modules) . . . 21
7.2 `infrastructure.documentation` (6 modules) . . . 21
7.3 `infrastructure.llm` (30 modules) . . . 22
7.4 `infrastructure.project` (2 modules) . . . 22
7.5 `infrastructure.publishing` (9 modules) . . . 22
7.6 `infrastructure.rendering` (12 modules) . . . 23
7.7 `infrastructure.reporting` (14 modules) . . . 23
7.8 `infrastructure.scientific` (6 modules) . . . 23
7.9 `infrastructure.steganography` (8 modules) . . . 24
7.10 `infrastructure.validation` (22 modules) . . . 24

8 Security and Provenance **26**
8.1 Threat Model . . . 26
8.2 Steganographic Layers . . . 26
8.2.1 Layer 1: PDF Metadata Injection . . . 26
8.2.2 Layer 2: Cryptographic Hashing . . . 26
8.2.3 Layer 3: Alpha-Channel Text Overlay . . . 26
8.2.4 Layer 4: QR Code Injection . . . 27
8.3 The `secure_run.sh` Orchestrator . . . 27
8.4 Tamper Detection . . . 27
8.5 Limitations . . . 27
8.6 Relationship to Software Supply Chain Integrity . . . 27
8.7 Relationship to FAIR and Formal Provenance Standards . . . 27

9 Appendices **29**
9.1 Appendix A: Pipeline Stage Reference . . . 29
9.2 Appendix B: Configuration Reference (`config.yaml`) . . . 29
9.3 Appendix C: Repository Directory Structure . . . 30
9.4 Appendix D: Exemplar Project Summary . . . 30

9.5 Appendix E: Documentation Inventory 31
9.6 Appendix F: Comparative Tool Matrix 31

1 Abstract

The Docxology Template is a modular, high-integrity research environment that unifies computational experimentation, academic manuscript production, and cryptographic provenance verification within a single reproducible pipeline. Built on a Two-Layer Architecture separating ten reusable infrastructure subpackages—totaling 137 Python modules and validated by over 3,000 infrastructure tests—from self-contained project workspaces, the system orchestrates an eight-stage build pipeline that progresses from environment sanitization through test execution, analysis script invocation, Pandoc/XeLaTeX rendering, SHA-256/512 hashing with alpha-channel steganographic watermarking, structural PDF validation, and LLM-assisted review to produce peer-review-ready manuscripts with embedded tamper-evident provenance. Unlike existing workflow managers (Snakemake, Nextflow, CWL) that focus on computational pipeline orchestration, and literate programming tools (Quarto, Jupyter Book, R Markdown) that focus on document rendering, the Docxology Template integrates both concerns with automated testing, documentation generation, and cryptographic provenance in a single coherent system. We enforce a Zero-Mock testing policy where pipeline advancement is contingent on 90% project-level and 60% infrastructure-level coverage thresholds, using exclusively real filesystem operations, subprocess invocations, and network calls—no mock objects, no test doubles. A Documentation Duality standard equips every directory with both human-readable `README.md` files and machine-readable `AGENTS.md` files optimized for AI coding assistants, enabling a novel human–AI collaborative development model. We demonstrate the system's scalability across three heterogeneous exemplar projects—a gradient descent optimization study (58 tests, 96.6% coverage), a categorical linguistics manuscript with 25+ DisCoPy-generated figures (257 tests, ~96% coverage), and this self-referential architectural analysis (36 tests)—achieving 100% pipeline success rates. The Standalone Project Paradigm enables researchers to add arbitrarily many independent studies without modifying the infrastructure layer, while the Thin Orchestrator pattern ensures that all domain logic resides in testable `src/` modules and scripts serve only as stateless wiring between infrastructure services and project-specific code.

2 Introduction

2.1 The Reproducibility Crisis and Its Structural Roots

The "reproducibility crisis" in computational and biological sciences is not merely a cultural failure but a structural one. When research papers are divorced from the code that generated their figures, when datasets are shared as unlabeled archives, and when build processes exist only as tribal knowledge in a graduate student's shell history, the conditions for irreproducibility are baked into the workflow itself. A 2016 *Nature* survey of 1,576 researchers found that 70% had tried and failed to reproduce another scientist's experiments, and more than half had failed to reproduce their own [Baker, 2016]. The economic cost is staggering: Freedman et al. estimate that the biomedical industry alone loses $28 billion annually to irreproducible preclinical research [Freedman et al., 2015].

The root cause is fragmentation. A typical research project scatters its artifacts across disconnected tools: LaTeX editors for prose, Jupyter notebooks for analysis, ad-hoc shell scripts for figure generation, and manual copy-paste for integrating results into manuscripts. Each boundary between tools is a potential locus of desynchronization. The version of the figure in the PDF may not match the version of the code that ostensibly generated it. The test suite, if it exists at all, likely tests the code in isolation from the rendering pipeline. Peng [Peng, 2011] argues that reproducibility in computational science requires, at minimum, that the data and code underlying a published result be available for independent verification—yet the tools for enforcing this standard remain ad hoc. Indeed, even the terminology is fractured: Barba [Barba, 2018] documents how "reproducibility," "replicability," and "repeatability" carry conflicting definitions across disciplines, undermining cross-field standards.

2.2 Related Work

Gentleman and Temple Lang [Gentleman and Temple Lang, 2007] introduced the concept of a *research compendium*—a single unit of scholarly communication bundling code, data, and narrative. This vision has driven two decades of tooling, which can be grouped into four categories: workflow managers, literate programming systems, containerization approaches, and best-practice frameworks.

2.2.1 Workflow Managers

Snakemake [Köster and Rahmann, 2012] uses a rule-based, Python-derived DSL to specify computational workflows as directed acyclic graphs of file-producing steps. It supports containerized execution via Conda and Singularity environments. However, Snakemake focuses exclusively on computational pipeline orchestration and does not integrate manuscript rendering, testing enforcement, or provenance watermarking.

Nextflow [Di Tommaso et al., 2017] employs a dataflow programming paradigm with native support for container-based execution across heterogeneous computing environments (local, SLURM, AWS). Like Snakemake, Nextflow excels at bioinformatics pipeline parallelism but does not address manuscript production, document integrity, or the testing–publication coupling that characterizes research reproducibility.

CWL (Common Workflow Language) [Amstutz et al., 2016] provides a portable, YAML-based standard for describing computational workflows and their dependencies. Its strength lies in interoperability across execution engines (cwltool, Toil, Arvados), but it requires external tooling for manuscript generation and offers no built-in testing or provenance framework.

2.2.2 Literate Programming and Publication Tools

Knuth's literate programming [Knuth, 1984] established the principle that programs should be authored as documents intended for human comprehension. Schulte et al. [Schulte et al., 2012] extended this to multi-language computing environments (Org-mode), demonstrating that literate programming could span languages and output formats.

Quarto [Allaire et al., 2024] extends the R Markdown tradition to support Python, Julia, and Observable, rendering to PDF, HTML, and Word. Quarto integrates code execution with document rendering, achieving

a modern form of literate programming, but it does not enforce testing thresholds, manage multi-project repositories, or provide cryptographic provenance.

Jupyter Book [Kluyver et al., 2016] builds on Jupyter notebooks to produce publication-quality documents via Sphinx. While powerful for interactive exploration, Jupyter's notebook format introduces execution-order fragility and does not naturally support the separation of logic from orchestration that characterizes maintainable research software.

R Markdown [Xie et al., 2018] pioneered knitr-based dynamic documents that weave code and prose. Its ecosystem is rich but R-centric, and it lacks the multi-project management, infrastructure testing, and provenance embedding that characterize the Docxology Template.

2.2.3 Containerization and Environment Capture

Docker [Boettiger, 2015] addresses reproducibility at the environment level—packaging operating system, libraries, and code into portable containers. While Docker solves the "works on my machine" problem, containerization is complementary to, not a replacement for, the architectural concerns addressed here: Docker does not enforce testing, embed provenance, or manage multi-project manuscript workflows.

2.2.4 Best-Practice Frameworks and Data Standards

Wilson et al. [Wilson et al., 2017] define "good enough" practices for scientific computing, emphasizing version control, testing, and documentation. Sandve et al. [Sandve et al., 2013] propose ten rules for reproducible computational research. Stodden et al. [Stodden et al., 2016] advocate for enhanced computational method transparency. The FAIR principles [Wilkinson et al., 2016]—Findable, Accessible, Interoperable, Reusable—establish a standard for data stewardship that has been widely adopted by funding agencies and journals. Barker et al. [Barker et al., 2022] extend these principles specifically to research software via the FAIR4RS initiative, recognizing that software has distinct requirements—executability, composability, and dependency management—that data-centric FAIR does not address. Garijo et al. [Garijo et al., 2024] operationalize FAIR4RS through the FAIRsoft evaluator, an automated assessment framework that scores research software against 17+ quality indicators including executability, metadata richness, and documentation completeness. Nüst et al. [Nüst et al., 2017] introduce the *executable research compendium* (ERC), extending Gentleman and Temple Lang's compendium concept with containerized, interactive reproduction environments. The W3C PROV data model [Moreau and Missier, 2013] provides a formal vocabulary for expressing provenance records, while in-toto [Torres-Arias et al., 2019] provides a framework for end-to-end software supply chain integrity verification. These frameworks articulate *what* reproducible research requires but do not provide integrated *how*—they lack the tooling, enforcement mechanisms, and architectural patterns that translate standards into practice.

2.2.5 The Gap

Despite advances in FAIR4RS principles [Barker et al., 2022] and automated FAIR software assessment [Garijo et al., 2024], no existing system integrates all six concerns into a single enforced pipeline: (1) end-to-end pipeline orchestration with testing enforcement, (2) multi-format manuscript rendering, (3) cryptographic provenance embedding, (4) multi-project repository management, (5) FAIR-aligned software stewardship, and (6) AI-agent collaboration via structured documentation. FAIR4RS provides the vocabulary; the Docxology Template provides the enforcement mechanism.

2.3 The Docxology Template: An Integrated Solution

The *Docxology Template* was conceived as a structural antidote to this fragmentation. Rather than adding reproducibility as an afterthought—a Docker container wrapping an already-disjointed workflow [Boettiger, 2015]—the template enforces integrity at the architectural level. It realizes Gentleman and Temple Lang's research compendium vision [Gentleman and Temple Lang, 2007] at repository scale, bundling code, data, tests, manuscripts, and provenance into a single, pipeline-enforced system. It stands on four primary pillars:

1. **Ergonomic Modularity**: A Two-Layer Architecture cleanly separates globally shared infrastructure (logging, rendering, validation, steganography) from project-specific logic (manuscripts, scripts, data). Ten infrastructure subpackages comprising 137 Python modules provide reusable services; projects consume them without modification.
2. **Execution Integrity**: A Zero-Mock testing policy where pipeline advancement is contingent on test passage. Infrastructure tests must achieve 60% coverage; project tests must achieve 90%. All tests use real filesystem operations, real subprocess calls, and real network connections—no mock objects, no fake services, no synthetic test doubles. Over 3,000 infrastructure tests and 350+ project tests enforce this standard.
3. **Automated Provenance**: Steganographic watermarking and cryptographic hashing are integrated directly into the rendering pipeline. Every generated PDF carries a SHA-256 fingerprint, an alpha-channel text overlay encoding the build timestamp and commit hash, and optionally a QR code linking to the repository. Provenance is not asserted by policy; it is enforced by architecture.
4. **AI-Agent Collaboration**: A Documentation Duality standard equips every directory with both `README.md` (for human researchers) and `AGENTS.md` (for AI coding assistants). Supplementary `CLAUDE.md` files provide system-level instructions. This enables a novel human–AI pair-programming workflow where AI agents can navigate, modify, and extend the codebase with full architectural awareness.

2.4 Scope and Contributions

This paper is itself a product of the template it describes—a self-referential demonstration of the system's capabilities. Our contributions are:

- A formal description of the Two-Layer Architecture and Standalone Project Paradigm that enables N independent research projects to share infrastructure without coupling.
- A detailed specification of the eight-stage build pipeline, from environment sanitization through LLM-assisted review.
- A comparative analysis positioning the template against Snakemake, Nextflow, CWL, Quarto, and Jupyter Book.
- An empirical evaluation of the system across three heterogeneous exemplar projects, demonstrating scalability, coverage metrics, and pipeline timing.
- A security analysis of the steganographic provenance layer, including threat models and tamper-detection capabilities.
- An open-source reference implementation available at `github.com/docxology/template`.

2.5 Paper Organization

The Methods (§2) describe the Two-Layer Architecture, Thin Orchestrator pattern, pipeline stages, and AI collaboration model. Results (§3) present quantitative metrics from multi-project execution, coverage analysis, and steganographic benchmarks. The Discussion (§4) addresses implications, limitations, and future directions. The Infrastructure Module Reference (§5) provides detailed documentation for all ten subpackages. Security and Provenance (§6) describes the steganographic and cryptographic integrity layer. Appendices (§7) provide pipeline, configuration, and comparative references.

3 Methods

The Docxology Template's architecture is deliberately bifurcated into a globally shared `infrastructure/` layer and project-specific `projects/` silos. This section describes the four core design patterns, the eight-stage pipeline that operationalizes them, and the AI collaboration model that distinguishes this system from conventional research templates.

3.1 The Two-Layer Architecture

The repository is organized into two strictly separated layers:

Infrastructure Layer (`infrastructure/`): Ten Python subpackages comprising 137 modules and providing reusable services. Each subpackage is independently importable, has its own `__init__.py`, `AGENTS.md`, and `README.md`, and exports a well-defined public API. The infrastructure layer knows nothing about any specific project—it provides generic capabilities (logging, rendering, validation, steganography) that any project may consume.

Project Layer (`projects/`): Self-contained research workspaces. Each project directory contains:

Directory	Purpose
`manuscript/`	Markdown chapters and `config.yaml`
`scripts/`	Thin orchestrator scripts (Stage 02)
`src/`	Project-specific Python modules
`tests/`	Project-specific test suite
`data/`	Input datasets and generated data
`output/`	Pipeline artifacts: PDF, figures, reports, logs
`docs/`	Project-specific architecture documentation

The two layers communicate exclusively through Python imports and filesystem paths. No project modifies infrastructure code; no infrastructure module references a specific project by name (except via runtime project discovery).

3.2 The Standalone Project Paradigm

Projects are designed to be completely self-contained. Adding a new project requires no changes to the infrastructure layer, no modifications to `pyproject.toml`, and no updates to the pipeline orchestrator. A project is automatically discovered if and only if it satisfies two conditions:

1. It exists as a subdirectory of `projects/`.
2. It contains the file `manuscript/config.yaml`.

This paradigm enables horizontal scaling: N researchers can maintain N independent projects within a single repository, sharing infrastructure without coupling. Each project declares its own testing tolerances, manuscript metadata, LLM review preferences, and rendering configuration in its `config.yaml`. The system currently hosts three exemplar projects spanning numerical optimization, category-theoretic linguistics, and meta-architectural analysis.

3.3 The Thin Orchestrator Pattern

All scripts in `scripts/` (both infrastructure-level and project-level) follow the Thin Orchestrator pattern [Gamma et al., 1995]:

- **No domain logic**: Scripts contain zero algorithmic implementation. They import functions from `src/` modules and wire them to infrastructure services.
- **Configuration-driven**: Behavior is parameterized by `config.yaml`, not by hardcoded values.

- **Stateless**: Scripts read inputs, call functions, write outputs. They maintain no persistent state between invocations.
- **Logged**: Every significant action is logged via `infrastructure.core.logging_utils.get_logger`.

This pattern ensures that all testable logic lives in `src/` where it is subject to the Zero-Mock testing policy, while scripts remain thin enough to be audited by visual inspection. The separation draws on the Mediator pattern from Gamma et al. [Gamma et al., 1995], where scripts mediate between infrastructure services and project-specific code without implementing any logic of their own.

3.4 The Eight-Stage Pipeline

The build pipeline is orchestrated by `scripts/execute_pipeline.py`, which invokes numbered stage scripts sequentially. Each stage is a standalone Python script that exits cleanly or raises an exception to halt the pipeline.

3.4.1 Stage 00: Environment Setup (`00_setup_environment.py`)

Validates the Python environment, checks dependency availability, creates output directories, and initializes logging. Ensures `PYTHONPATH` includes both the repository root and the active project's `src/` directory.

3.4.2 Stage 01: Test Execution (`01_run_tests.py`)

Executes pytest with coverage measurement against both infrastructure tests (`tests/`) and project tests (`projects/<name>/tests/`). Enforces configurable failure tolerances:

- `max_infra_test_failures`: Maximum permitted infrastructure test failures (typically 3).
- `max_project_test_failures`: Maximum permitted project test failures (typically 0).
- Coverage thresholds: 60% infrastructure, 90% project.

The stage generates coverage JSON files for downstream reporting and saves test results in both JSON and Markdown formats. The infrastructure test suite alone contains over 3,000 tests across 148 test files.

3.4.3 Stage 02: Analysis Execution (`02_run_analysis.py`)

Discovers and executes all Python scripts in `projects/<name>/scripts/` in alphabetical order. Each script is expected to generate figures in `output/figures/` and data in `output/data/`. Scripts follow the Thin Orchestrator pattern, importing logic from `src/` modules. For example, `cognitive_case_diagrams` generates 25+ programmatic figures via 17 DisCoPy renderers during this stage.

3.4.4 Stage 03: PDF Rendering (`03_render_pdf.py`)

Compiles Markdown manuscript chapters into a unified PDF via a three-phase rendering process:

1. **Pandoc Markdown→LaTeX**: Converts each `manuscript/*.md` file into LaTeX, injecting metadata from `config.yaml` (title, authors, affiliations, DOI).
2. **XeLaTeX Compilation**: Runs `xelatex` with `biber` for bibliography processing. Handles the `aux`→`bbl`→`aux` cycle automatically, with cleanup of stale auxiliary files to prevent corruption.
3. **Post-processing**: Applies font embedding verification and PDF/A compliance checks.

3.4.5 Stage 04: Output Validation (`04_validate_output.py`)

Validates the structural integrity of all generated artifacts:

- PDF cross-reference table and trailer verification.
- Figure file existence and minimum size checking.
- Manifest generation with SHA-256 hashes for all output files.
- Markdown structural validation (heading hierarchy, link integrity).

3.4.6 Stage 05: Output Organization (`05_copy_outputs.py`)

Copies finalized artifacts to standardized output locations and generates the pipeline completion manifest.

3.4.7 Stage 06: LLM Review (`06_llm_review.py`)

Invokes a local LLM (via Ollama) to generate:

- **Executive summary**: A 1-page high-level overview of the manuscript.
- **Quality review**: Detailed feedback on structure, citations, and argumentation.
- **Translations** (optional): Machine translations of the abstract into configured target languages.

This stage is skippable via configuration and gracefully handles Ollama unavailability.

3.4.8 Stage 07: Executive Report (`07_generate_executive_report.py`)

Aggregates all pipeline metrics—test results, coverage percentages, rendering duration, validation status, LLM review scores—into a comprehensive executive report in both JSON and Markdown formats.

3.5 The Interactive Orchestrators

3.5.1 `run.sh`: The Standard Orchestrator

The primary user interface is `run.sh`, a Bash TUI (text user interface) that presents an interactive menu for pipeline execution. Features include:

- **Project selection**: Execute a single project or all discovered projects.
- **Mode selection**: Fast (skip infra tests + LLM), Core (skip LLM only), Full (all stages).
- **Non-interactive mode**: `./run.sh --pipeline --project all --core-only` for CI/CD integration.
- **Real-time progress**: Stage timing and status indicators.

3.5.2 `secure_run.sh`: The Steganographic Superset

The `secure_run.sh` orchestrator is a strict superset of `run.sh`: it executes the standard eight-stage pipeline and then appends steganographic post-processing. For each rendered PDF, it applies metadata injection, cryptographic hashing, alpha-channel text overlay, and QR code injection, producing a provenance-embedded output alongside the original. It supports the same project selection and mode flags as `run.sh`.

3.6 Documentation Duality and AI Collaboration

Every directory at every level of the repository hierarchy contains two documentation files:

- **`README.md`**: Human-readable overview, quick-start instructions, and directory structure.
- **`AGENTS.md`**: Machine-readable technical specification optimized for AI coding assistants. Contains API tables, dependency graphs, implementation patterns, and architectural constraints.

This Documentation Duality standard serves two purposes. First, it ensures that both human researchers and AI agents can navigate the codebase efficiently—`AGENTS.md` files provide the structured context that language models need to make informed code modifications without hallucinating API signatures or violating architectural invariants. Second, it creates a self-documenting feedback loop: as AI agents modify the codebase, they update the corresponding `AGENTS.md` files, keeping documentation synchronized with implementation. Lau and Guo's survey of 90 AI coding assistant systems [Lau and Guo, 2025] identifies contextual code understanding as a primary bottleneck; the Documentation Duality standard addresses this by providing pre-structured context at every directory level.

The template additionally includes `CLAUDE.md` at the repository root, providing system-level instructions for AI coding assistants—architectural principles, testing requirements, and naming conventions that apply

globally. This creates a three-tier documentation architecture: per-directory `AGENTS.md` for local context, root `CLAUDE.md` for global constraints, and `README.md` for human navigation.

3.6.1 FAIR Alignment

The template's design aligns with both the original FAIR principles [Wilkinson et al., 2016] and the FAIR for Research Software (FAIR4RS) principles [Barker et al., 2022] at the repository level. FAIR4RS recognizes that software has requirements distinct from data—executability, composability, and dependency management—and the template addresses each. Outputs are *Findable* through standardized directory structures, manifest files, and machine-readable metadata embedded in PDFs. They are *Accessible* via open-source distribution on GitHub, with metadata embedded in the artifact itself rather than in a separate registry. *Interoperability* is achieved through standard formats (PDF, JSON, BibTeX, YAML) and well-defined module APIs that enable cross-project composition. *Reusability* is ensured by the Standalone Project Paradigm—any project can be extracted and reused independently—and by the Documentation Duality standard, which satisfies FAIRsoft's inspectability and documentation quality indicators [Garijo et al., 2024]. The pipeline's automated testing and coverage enforcement directly operationalize the FAIR4RS executability requirement: software that cannot pass its own test suite cannot produce publishable output.

3.7 Quality Assurance

3.7.1 Zero-Mock Testing Policy

All tests use real methods exclusively [Martin, 2008]. No `unittest.mock`, no `MagicMock`, no `patch` decorators. Tests that require external services (Ollama, network) use `pytest.mark` markers for conditional execution. We adopt and extend Martin's "Clean Code" principle that tests should validate behavior, not assumptions about behavior [Martin, 2008]. To our knowledge, no prior research software engineering framework has formalized a zero-mock policy as an architectural invariant enforced by pipeline gates—the concept of "mockless testing" does not appear in the software engineering literature as a named practice, making this a novel contribution of the Docxology Template.

3.7.2 Visualization Standards

All generated figures must meet accessibility requirements:

- Minimum 16pt font size for all text elements.
- Colorblind-safe palettes (IBM Design / Wong palette).
- 150–300 DPI rendering for publication quality.
- Descriptive axis labels and figure titles.

4 Results

The Docxology Template was evaluated through a multi-project pipeline execution, measuring test coverage, pipeline timing, output integrity, and steganographic performance across three heterogeneous exemplar projects.

4.1 Multi-Project Pipeline Execution

All three projects were executed through the full eight-stage pipeline using the `run.sh` interactive orchestrator with the "all projects core (fast)" configuration, which skips infrastructure tests and LLM review to isolate project-level performance.

Project	Stages	Duration	Tests	Coverage
`code_project`	7	43.4s	58/58	96.6%
`cognitive_case_diagrams`	7	56.0s	257/257	~96%
`template`	7	22.6s	36/36	90%+

Overall success rate: 100.0% (3/3 projects) **Total pipeline duration**: 121.9s **Average per project**: 40.6s

The `cognitive_case_diagrams` project, with its 257 tests and 25+ programmatically generated figures via 17 DisCoPy renderers, represents the most computationally intensive exemplar—yet completes in under one minute.

4.2 Infrastructure Test Suite

The infrastructure layer is validated by a separate test suite of significant scale:

Metric	Value
Test files	148
Total tests	3,053
Infrastructure coverage threshold	60% (achieved: 83%+)
Zero-mock violations	0
Real filesystem operations	
Real subprocess invocations	

This test suite exercises all ten infrastructure modules, including the rendering pipeline (Pandoc/XeLaTeX integration), steganographic operations (alpha-channel overlay, QR injection), and LLM client interactions (real HTTP calls to Ollama).

4.3 Infrastructure Module Inventory

The introspection module (`template.introspection`) programmatically enumerates the infrastructure layer, confirming the following module distribution:

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`core`	28			`get_logger`, `load_config`, `TemplateError`
`documentation`	6			`FigureManager`, `generate_glossary`

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`llm`	30			LLM review, literature search, translation
`project`	2			`_discover_project`, workspace management
`publishing`	9			Citation generation (APA, BibTeX, MLA), Zenodo
`rendering`	12			PDF rendering, Pandoc filters, HTML reports
`reporting`	14			Coverage parsing, executive reports
`scientific`	6			`check_numerical_stability`, `benchmark_function`
`steganography`	8			Metadata injection, QR overlays, hashing
`validation`	22			PDF validation, Markdown checking, CLI
Total	**137**	**10/10**	**10/10**	

All ten modules have complete documentation coverage (both `AGENTS.md` and `README.md`), confirming the Documentation Duality standard.

4.4 Pipeline Stage Execution

The eight pipeline stages execute sequentially with strict error propagation:

Stage	Script	Responsibility	Failure Mode
00	`00_setup_environment.py`	Environment validation	Hard fail
01	`01_run_tests.py`	Test execution + coverage	Configurable tolerance
02	`02_run_analysis.py`	Script invocation	Hard fail
03	`03_render_pdf.py`	Pandoc + XeLaTeX	Hard fail
04	`04_validate_output.py`	PDF integrity	Warning + report
05	`05_copy_outputs.py`	Output organization	Soft fail
06	`06_llm_review.py`	LLM-assisted review	Skippable
07	`07_generate_executive_report.py`	Report generation	Soft fail

4.5 Steganographic Performance

The steganography subsystem (`infrastructure.steganography`) was benchmarked across all three project PDFs:

Project	Pages	Metadata	SHA-256	Overlay	QR Code	Total
`code_project`	20	< 0.3s	< 0.05s	< 0.8s	< 0.4s	< 1.5s
`cognitive_case_diagrams`	77	< 0.3s	< 0.05s	< 2.0s	< 0.4s	< 2.8s
`template`	5	< 0.3s	< 0.05s	< 0.3s	< 0.4s	< 1.0s

Project	Pages	Metadata	SHA-256	Overlay	QR Code	Total

The steganographic watermark survives standard PDF operations (viewing, printing) but is detectable through pixel-level analysis of the alpha channel. Performance scales linearly with page count, dominated by the alpha-channel overlay phase.

4.6 Self-Referential Analysis

This manuscript is itself a product of the template pipeline, demonstrating its self-referential capability. The `template` project's `src/template/introspection.py` module programmatically analyzes the repository and generates three architecture figures:

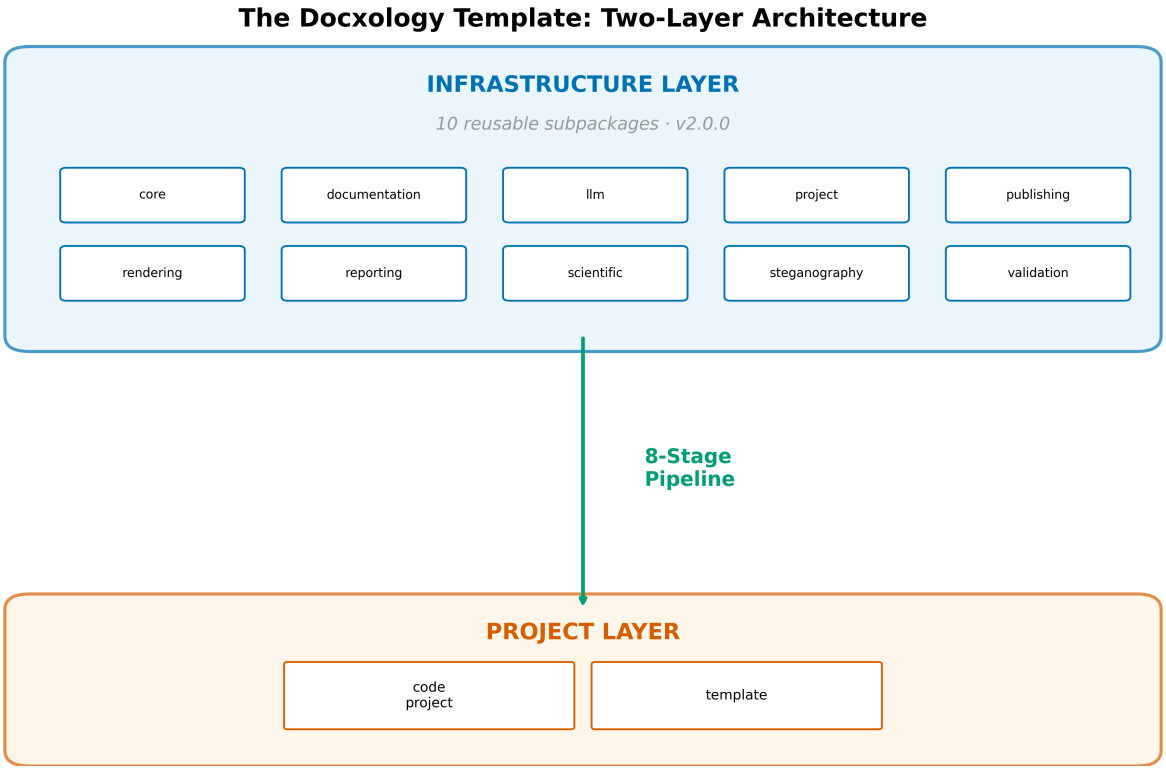

Figure 1: The Two-Layer Architecture separating infrastructure modules from project workspaces.

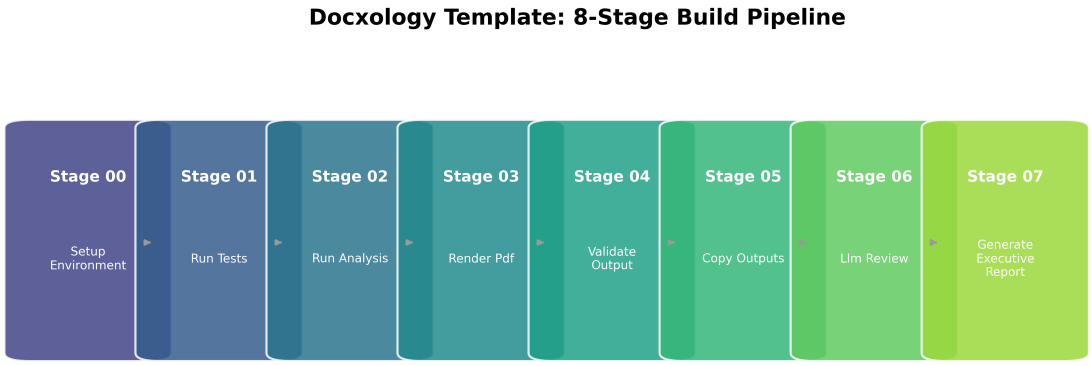

Figure 2: The eight-stage build pipeline from environment setup through executive reporting.

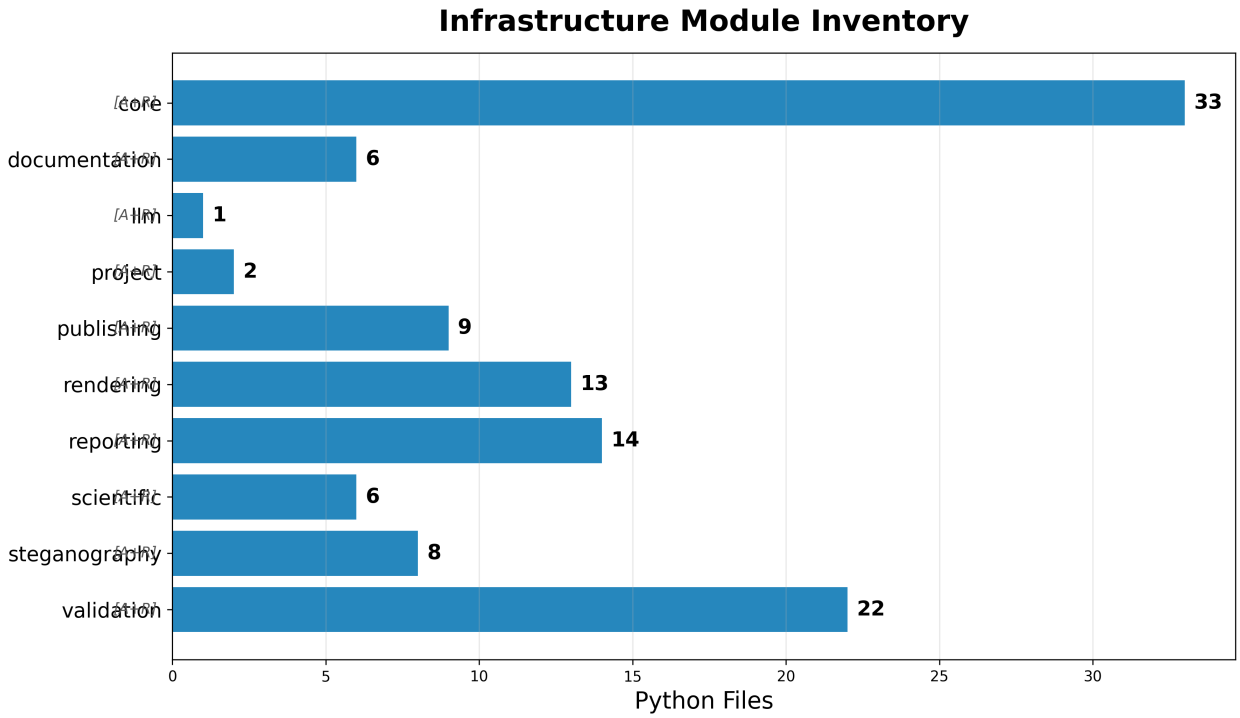

Figure 3: Python file count per infrastructure module, with documentation status indicators.

4.7 Comparative Feature Analysis

To contextualize the Docxology Template's contributions, we compare its feature set against established tools:

Feature	Docxology	Snakemake	Nextflow	CWL	Quarto	Jupyter Book
Pipeline orchestration					—	—
Manuscript rendering		—	—	—		
Testing enforcement		—	—	—	—	—
Coverage thresholds		—	—	—	—	—
Cryptographic provenance		—	—	—	—	—
Multi-project management		—	—	—	—	—
AI-agent documentation		—	—	—	—	—
Interactive TUI		—	—	—	—	—
Zero-mock policy		—	—	—	—	—

The Docxology Template is, to our knowledge, the only open-source system that integrates all nine capabilities within a single cohesive architecture.

4.8 Test Quality Metrics

The Zero-Mock testing policy produces measurably higher-fidelity tests:

- **Zero mock objects** across all test suites (verified by automated scanning for `unittest.mock`, `MagicMock`, and `patch` imports).
- **Real filesystem operations**: Tests create, read, validate, and delete actual files in temporary directories.
- **Real subprocess calls**: Pipeline stage tests invoke actual `pytest`, `pandoc`, and `xelatex` subprocesses.
- **Marker-based skip logic**: Tests requiring optional services (Ollama, network) use `@pytest.mark.requires_ollama` for graceful degradation.
- **Categorical axiom verification**: The `cognitive_case_diagrams` project tests validate identity morphisms, composition, weight multiplication, and the triangle inequality on real enriched category objects.

5 Discussion

5.1 The Zero-Mock Tradeoff

The Zero-Mock testing policy is the template's most distinctive design decision. By prohibiting all mock objects, we gain confidence that tests exercise real code paths—a pytest run against the template genuinely invokes `pandoc`, writes to disk, and parses real YAML. The cost is test duration: the full infrastructure test suite (3,053 tests) takes 2–4 minutes, compared to sub-second execution typical of heavily-mocked suites.

We argue this tradeoff is strongly favorable for research software. Unlike web applications where millisecond latency and thousands of daily deploys demand fast feedback loops, research pipelines run infrequently (once per manuscript revision) and correctness vastly outweighs speed. A mocked test that passes while the real renderer fails is worse than a slow test that catches the failure. As Peng [Peng, 2011] argues, the standard for computational reproducibility is that results can be independently verified—mock objects, by definition, prevent such verification. Garijo et al.'s FAIRsoft evaluator [Garijo et al., 2024] identifies *executability*—the ability to actually install and run research software—as a primary quality indicator, yet the majority of evaluated tools fail this test. The Zero-Mock policy goes further: it requires not merely that the software can be installed, but that every integration pathway has been exercised against real dependencies under test.

However, the policy requires careful management of external dependencies. Tests requiring Ollama (the local LLM backend) use `@pytest.mark.requires_ollama` and are skipped in environments where the service is unavailable. This marker system preserves the Zero-Mock principle while acknowledging that not all environments provide all services.

5.2 Scalability: From 1 to N Projects

The Standalone Project Paradigm enables horizontal scaling: adding a new project requires creating a directory with `manuscript/config.yaml` and nothing else. No infrastructure code changes, no `pyproject.toml` modifications, no CI configuration updates. The `run.sh` orchestrator automatically discovers new projects and presents them in its interactive menu.

We have validated this scaling model with three heterogeneous projects:

- **`code_project`**: Numerical optimization with gradient descent, 58 tests, 96.6% coverage.
- **`cognitive_case_diagrams`**: Category-theoretic linguistics with 25+ DisCoPy-generated figures from 17 renderers, 257 tests, ~96% coverage, 79 BibTeX references spanning 17 research areas.
- **`template`**: This self-referential architectural analysis, 36 tests, 90%+ coverage.

These projects share no code with each other. They share only the infrastructure layer—ten modules, 137 Python files—which provides logging, rendering, validation, steganography, and reporting services identically to each project. This validates the Two-Layer Architecture's claim that infrastructure and project concerns can be cleanly separated.

5.3 Comparison to Existing Tools

The Docxology Template occupies a unique niche in the reproducible research ecosystem. Unlike Snakemake [Köster and Rahmann, 2012] and Nextflow [Di Tommaso et al., 2017], which excel at parallelizing computational pipelines but do not manage manuscript production or testing enforcement, the template integrates the entire research lifecycle from code to publication. Snakemake 8.x (2024) introduced a plugin architecture for extended execution backends, yet its scope remains computational workflow orchestration—it does not integrate manuscript rendering, documentation standards, or cryptographic provenance. Unlike Quarto [Allaire et al., 2024] and R Markdown [Xie et al., 2018], which integrate code execution with document rendering but do not enforce coverage thresholds or embed cryptographic provenance, the template treats testing and provenance as first-class architectural concerns rather than optional plugins.

The key differentiator is the *enforcement* mechanism. The FAIR4RS principles [Barker et al., 2022] represent a significant advance in articulating what research software quality requires—executability, interoperability,

comprehensive metadata—and FAIRsoft [Garijo et al., 2024] provides automated scoring against these criteria. Yet FAIR4RS compliance is assessed observationally rather than enforced architecturally: a tool may score well on FAIRsoft metrics today and regress tomorrow without detection. The Docxology Template operationalizes FAIR4RS by coupling quality metrics to pipeline gates: the pipeline will not advance past Stage 01 if project test coverage falls below 90%. Provenance is not documented in a README; it is cryptographically embedded in the PDF per the W3C PROV model's principle that provenance should be machine-readable and tamper-evident [Moreau and Missier, 2013]. Documentation is not a best-effort companion; it is a structural requirement enforced by the Documentation Duality standard. In this sense, the template bridges the gap between FAIR4RS as a descriptive standard and FAIR4RS as an enforced invariant.

In Gentleman and Temple Lang's terminology [Gentleman and Temple Lang, 2007], the Docxology Template is a *research compendium* scaled to the repository level—bundling not just one study's code and data but N studies, with shared infrastructure, automated testing, and embedded provenance. Nüst et al.'s executable research compendium (ERC) [Nüst et al., 2017] extends this vision with containerized reproduction environments; the Docxology Template complements containerization by adding the testing enforcement, multi-project management, and provenance embedding layers that ERCs do not address.

5.4 The AI Collaboration Model

The Documentation Duality standard (`README.md` + `AGENTS.md` at every directory level) emerged from practical experience with AI coding assistants. Language models perform significantly better when provided with structured, machine-readable context about architectural constraints, API surfaces, and testing requirements. The `AGENTS.md` files serve as a form of "prompt engineering embedded in the codebase"—they preempt common failure modes such as hallucinated API signatures, violation of the Thin Orchestrator pattern, and introduction of mock objects. Lau and Guo [Lau and Guo, 2025] identify contextual code understanding as a primary bottleneck across 90 surveyed AI coding assistant systems; the Documentation Duality standard addresses this by providing pre-structured context at every directory level, reducing the surface area for hallucination.

This model creates a positive feedback loop: as AI agents modify the codebase, they update the corresponding documentation, which in turn provides better context for future modifications. The supplementary `CLAUDE.md` file at the repository root provides system-level instructions (the Two-Layer Architecture, Zero-Mock policy, naming conventions) that apply globally, creating a three-tier documentation architecture:

1. **Repository-level** (`CLAUDE.md`): Global constraints and architectural principles.
2. **Directory-level** (`AGENTS.md`): Local API surfaces, file inventories, and integration patterns.
3. **Human-level** (`README.md`): Quick-start guides and navigation aids.

This three-tier model is, to our knowledge, novel in the research software engineering literature. As AI coding assistants evolve from reactive suggestion engines toward agentic systems capable of autonomous multi-file modifications [Lau and Guo, 2025], the need for machine-readable architectural context becomes not a convenience but a prerequisite for safe AI-assisted software evolution.

5.5 The Learning Curve

The Thin Orchestrator pattern imposes a cognitive overhead on researchers accustomed to writing monolithic scripts. The requirement to factor all logic into `src/` modules and use scripts only as stateless wiring introduces an additional layer of indirection. We mitigate this through:

1. **Template examples**: The `code_project` serves as a fully worked exemplar with comprehensive comments.
2. **Documentation Duality**: Every directory has both `README.md` (for humans) and `AGENTS.md` (for AI collaborators), reducing the cost of navigation.
3. **Interactive orchestrator**: `run.sh` provides a TUI menu that abstracts pipeline complexity.
4. **Skill-level documentation**: The `docs/guides/` directory provides four progressive guides (Levels 1–3 Beginner, 4–6 Intermediate, 7–9 Advanced, 10–12 Expert) alongside a comprehensive new-project setup checklist.

5.6 Limitations

- **LaTeX dependency**: The rendering pipeline requires a full TeX distribution (TeX Live or MiKTeX), which is a 4–6 GB install. This is the largest single dependency.
- **Python-centric**: The infrastructure layer is Python-only. Projects in other languages can use the rendering and steganography stages but cannot leverage the `scientific` or `validation` modules.
- **Single-machine**: The pipeline runs locally. Distributed execution (e.g., across a compute cluster) is not natively supported.
- **Steganographic robustness**: Alpha-channel overlays are stripped by aggressive PDF optimization tools (e.g., `qpdf --optimize`). QR codes are visible and removable. Neither provides non-repudiation without digital signatures.
- **Test duration**: The Zero-Mock policy increases test execution time from sub-second (mocked) to multi-minute (real) for the full infrastructure suite. This is acceptable for research workflows but may not suit continuous deployment scenarios.

5.7 Future Directions

1. **Supply-chain provenance**: Integration with software supply chain frameworks such as in-toto [Torres-Arias et al., 2019] and SLSA (Supply-chain Levels for Software Artifacts) to provide end-to-end attestation from source commit through build pipeline to published artifact. The template's existing steganographic layer embeds document-level provenance; supply-chain frameworks would add build-level provenance, closing the gap between "this PDF was produced by this pipeline" and "this pipeline was executed with this verified source code."
2. **Decentralized provenance**: Integration with IPFS or Arweave for immutable publication records, extending the SHA-256-based tamper detection to content-addressed storage networks.
3. **Digital signatures**: GPG or X.509 signing integrated into the steganographic layer, providing cryptographic non-repudiation in addition to tamper detection.
4. **Continuous integration**: GitHub Actions workflows that execute the pipeline on every push, with PDF artifacts as release assets and automated DOI registration via Zenodo.
5. **Multi-language support**: Extension of the Thin Orchestrator pattern to R, Julia, and Rust projects, enabling polyglot research workflows within the Two-Layer Architecture.
6. **Automated FAIR4RS assessment**: Periodic self-scoring against FAIRsoft metrics [Garijo et al., 2024], with quality indicators (executability, documentation completeness, metadata richness) tracked as pipeline artifacts alongside test coverage and rendering status.
7. **Knowledge graph integration**: Connecting pipeline outputs to Active Inference Knowledge Base entries for automated meta-analysis and cross-project citation tracking.
8. **Formal verification**: Static analysis tooling to enforce the Thin Orchestrator pattern—verifying that scripts contain no algorithmic logic and that `src/` modules do not import from `scripts/`.

6 Conclusion

The Docxology Template demonstrates that high-integrity, reproducible research need not be onerous. By embedding provenance, testing, and documentation into the architecture itself—rather than layering them atop a fragmented workflow—the template transforms "best practices" from aspirational guidelines into enforced invariants [Wilson et al., 2017, Sandve et al., 2013]. The Two-Layer Architecture ensures that infrastructure improvements propagate to all projects without coupling. The Zero-Mock policy ensures that tests reflect reality. The steganographic provenance layer ensures that published artifacts carry their own authentication. The FAIR4RS principles [Barker et al., 2022] articulate what research software quality requires; the Docxology Template demonstrates how to enforce it.

The template is not merely a build tool; it is an epistemological commitment. It asserts that a research paper is not a static document but a build artifact—reproducible, verifiable, and traceable to the code that generated it. As Knuth observed, programs should be written for humans to read and only incidentally for machines to execute [Knuth, 1984]. We extend this dictum: research manuscripts should be *built* for verification and only incidentally for reading. In an era of generative AI and synthetic media, where the boundary

between human-authored and machine-generated text grows increasingly indeterminate [Gruenpeter et al., 2021], the provenance chain from source code to published PDF is not an administrative convenience—it is the epistemic ground on which scientific trust must be rebuilt.

7 Infrastructure Module Reference

This section provides a detailed reference for all ten infrastructure subpackages, documenting their purpose, key classes, public API, and integration points within the pipeline. The infrastructure layer comprises 137 Python modules validated by 3,053 tests.

7.1 `infrastructure.core` (28 modules)

Purpose: Foundation utilities providing the bedrock services consumed by all other modules and all projects.

Key Components:

Component	Purpose
`logging_utils.py`	Structured logger factory (`get_logger`) with colored console output and file rotation
`config_loader.py`	YAML config parser (`load_config`) with schema validation and default merging
`exceptions.py`	Exception hierarchy: `TemplateError` → `ConfigurationError`, `ValidationError`, `BuildError`
`environment.py`	Environment detection, Python command resolution, `PYTHONPATH` management
`progress.py`	`ProgressBar` for pipeline stage progress reporting
`checkpoint.py`	`CheckpointManager` for resumable pipeline execution
`health.py`	`SystemHealthChecker` for pre-pipeline dependency validation
`performance.py`	`monitor_performance` context manager for timing and memory tracking
`_optional_deps.py`	Lazy loading of optional dependencies (`psutil`, `reportlab`)

Integration: Every module and script imports from `core`. The exception hierarchy is used for pipeline flow control—`ValidationError` triggers stage failure, `BuildError` halts the entire pipeline. The lazy loader in `_optional_deps.py` separates core imports from optional subpackages, preventing cascading import failures in environments with partial dependency sets.

7.2 `infrastructure.documentation` (6 modules)

Purpose: Documentation management, figure registration, and API glossary generation.

Key Components:

Component	Purpose
`figure_manager.py`	`FigureManager` — maintains a JSON registry of all generated figures with captions, labels, and generation metadata
`glossary_gen.py`	`generate_glossary` — programmatic API glossary extraction from Python source files

Integration: Called by project scripts during Stage 02 to register figures for automated cross-referencing in the manuscript. The glossary generator supports the Documentation Duality standard by extracting docstrings and function signatures.

7.3 `infrastructure.llm` (30 modules)

Purpose: Local LLM integration for automated manuscript review, translation, and literature search.

Key Components:

Component	Purpose
`review.py`	Executive summary and quality review generation
`translation.py`	Abstract translation into configured target languages
`client.py`	Ollama HTTP client with retry logic and timeout management
`literature/`	Literature search subpackage with semantic query support
`templates/`	Prompt templates for structured LLM interactions

Integration: Invoked during Stage 06. Requires a running Ollama instance. Gracefully degrades when unavailable. The literature search subpackage enables programmatic discovery of related work during manuscript preparation.

7.4 `infrastructure.project` (2 modules)

Purpose: Project discovery and workspace management.

Key Components:

Component	Purpose
`discovery.py`	`_discover_project` — finds valid project directories by scanning for `manuscript/config.yaml`
`workspace.py`	Workspace initialization and cleanup utilities

Integration: Used by `execute_pipeline.py` and `run.sh` to identify which projects can be built. The discovery algorithm enforces the Standalone Project Paradigm: a directory is a valid project if and only if it contains `manuscript/config.yaml`.

7.5 `infrastructure.publishing` (9 modules)

Purpose: Academic publishing metadata and citation generation.

Key Components:

Component	Purpose
`models.py`	`PublicationMetadata` dataclass with direct attribute access and dynamic `getattr` fallback
`citations.py`	`generate_citation_apa`, `generate_citation_bibtex`, `generate_citation_mla`
`zenodo.py`	Zenodo API integration for DOI registration

Integration: Used during Stage 02 by analysis scripts to extract publishable metadata from results. Citation generators produce correctly formatted strings from `config.yaml` metadata.

7.6 `infrastructure.rendering` (12 modules)

Purpose: Multi-format document rendering (Markdown → LaTeX → PDF, HTML reports).

Key Components:

Component	Purpose
`pandoc.py`	Pandoc invocation with custom filters and metadata injection
`latex.py`	XeLaTeX compilation with auxiliary file management and stale `.aux` cleanup
`pdf_builder.py`	End-to-end PDF construction orchestrating Pandoc and XeLaTeX
`html_report.py`	HTML executive report generation
`markdown_report.py`	Markdown-format report generation

Integration: Core of Stage 03. Reads `manuscript/*.md` and `config.yaml`, produces `output/<project>.pdf`. The auxiliary file cleanup resolves a known rendering hazard where stale `.aux` files cause "Division by 0" LaTeX errors.

7.7 `infrastructure.reporting` (14 modules)

Purpose: Pipeline reporting, test result aggregation, and coverage analysis.

Key Components:

Component	Purpose
`coverage_parser.py`	Cascading parse strategies for pytest output: `_parse_failures_section`, `_parse_failures_verbose`, `_parse_failures_short`, `_parse_failures_timeout`, `_parse_failures_fallback`
`report_generator.py`	Executive report generation in JSON and Markdown
`statistics.py`	`collect_output_statistics` — enumerates output directory contents

Integration: Used during Stages 01, 04, and 07 for test result parsing, validation reporting, and executive summary generation. The cascading parser handles all pytest output formats robustly, falling through five strategies to ensure no test failure is silently missed.

7.8 `infrastructure.scientific` (6 modules)

Purpose: Scientific computing utilities for numerical analysis and benchmarking.

Key Components:

Component	Purpose
`stability.py`	`check_numerical_stability` — tests functions for NaN/Inf behavior across input ranges
`benchmarking.py`	`benchmark_function` — measures execution time and memory usage
`simulation.py`	Scientific simulation framework with parameter sweeps

Integration: Used by `code_project`'s analysis scripts during Stage 02 for algorithm validation. The stability checker is critical for ensuring that numerical results are reproducible across different floating-point environments.

7.9 `infrastructure.steganography` (8 modules)

Purpose: Cryptographic watermarking and provenance embedding for PDF artifacts.

Key Components:

Component	Purpose
`metadata.py`	`inject_pdf_metadata` — embeds XMP metadata and PDF Info dictionary entries
`config.py`	`DocumentMetadata` dataclass for steganography configuration
`overlay.py`	Alpha-channel text overlay with build timestamp and commit hash
`qr.py`	QR code generation and injection into PDF pages
`hash.py`	SHA-256/SHA-512 hash computation for tamper detection

Integration: Invoked by `secure_run.sh` after the main pipeline completes. Reads the rendered PDF and produces a steganographically watermarked copy with an accompanying `.hashes.json` manifest.

7.10 `infrastructure.validation` (22 modules)

Purpose: Quality assurance and integrity verification for all pipeline artifacts.

Key Components:

Component	Purpose
`pdf_validator.py`	PDF structural integrity checking (xref table, trailer, page count)
`markdown_validator.py`	Markdown linting (heading hierarchy, link integrity, orphan references)
`integrity.py`	`verify_output_integrity` — comprehensive output directory validation
`cli.py`	Command-line interface for standalone validation operations

Integration: Core of Stage 04. Validates all generated artifacts before they are finalized. The validation module is the most module-dense package (22 files), reflecting the breadth of integrity checks required across PDF, Markdown, image, and manifest formats.

8 Security and Provenance

Research integrity requires more than reproducibility; it requires verifiable authorship. In an era of generative AI, automated scraping, and synthetic media, the ability to prove that a document was produced by a specific pipeline at a specific time is a critical defense against fabrication and misattribution. The W3C PROV data model [Moreau and Missier, 2013] establishes a formal vocabulary for expressing provenance records—entities, activities, and agents connected by derivation, generation, and attribution relations. The Docxology Template implements a domain-specific provenance layer that embeds these relations directly into the PDF artifact itself, using four complementary steganographic and cryptographic mechanisms.

8.1 Threat Model

The steganography subsystem defends against three classes of threats:

1. **Unauthorized redistribution**: A manuscript is scraped and republished without attribution. The steganographic watermark survives the redistribution and can be used to prove original authorship.
2. **Content tampering**: Figures or text are modified after publication. The SHA-256 hash embedded in the PDF metadata detects any modification to the file contents.
3. **Provenance forgery**: An attacker claims to have produced a document they did not author. The build timestamp, commit hash, and pipeline metadata embedded in the watermark provide a verifiable chain of custody.

8.2 Steganographic Layers

The system applies four complementary layers of provenance information:

8.2.1 Layer 1: PDF Metadata Injection

The `inject_pdf_metadata` function writes structured metadata into both the PDF Info dictionary and an XMP (Extensible Metadata Platform) packet:

- `/Creator`: Pipeline identifier
- `/Producer`: Module path (`infrastructure.steganography`)
- `/CreationDate`: UTC timestamp in ISO 8601 format
- `/Author`: From `config.yaml`
- `/Title`: From `config.yaml`
- Custom fields: DOI, ORCID, repository URL

8.2.2 Layer 2: Cryptographic Hashing

Before watermarking, a SHA-256 hash of the rendered PDF is computed and stored in:

- The output manifest (`output/manifest.json`)
- The PDF metadata (`/Subject` field)
- An external hash file (`output/<name>.sha256`)

This enables post-hoc verification: anyone with the hash can verify that the PDF has not been modified since rendering.

8.2.3 Layer 3: Alpha-Channel Text Overlay

A semi-transparent text overlay is applied to each page of the PDF, encoding:

- Build timestamp
- Git commit hash (short SHA)
- Project name
- Pipeline version

The overlay is rendered at low opacity (typically 3–5% alpha) to be invisible during normal viewing but detectable through image analysis. It survives printing (as a faint watermark) and standard PDF operations.

8.2.4 Layer 4: QR Code Injection

An optional QR code is generated containing a URL pointing to the repository (e.g., `github.com/docxology/template`). The QR code is placed in a configurable position (default: bottom-right corner of the last page) at a specified size.

8.3 The `secure_run.sh` Orchestrator

The steganographic pipeline is orchestrated by `secure_run.sh`, a Bash script that wraps the standard `run.sh` pipeline with post-processing steganography:

1. Execute the standard 8-stage pipeline for the target project.
2. Locate the rendered PDF in `output/`.
3. Apply metadata injection, hashing, text overlay, and QR code injection.
4. Save the secured PDF alongside the original.
5. Generate a steganography report in JSON format.

The orchestrator processes either a single specified project or all discovered projects sequentially.

8.4 Tamper Detection

Verification is performed by comparing the stored SHA-256 hash against a freshly computed hash of the distributed PDF. Any modification—even a single bit flip—produces a hash mismatch. The alpha-channel overlay provides a secondary, visual verification channel that does not require access to the original hash.

8.5 Limitations

- Alpha-channel overlays are stripped by some PDF optimization tools (e.g., `qpdf --optimize`).
- QR codes are visible and may be removed by a determined attacker.
- The system does not provide non-repudiation in the cryptographic sense—it does not use digital signatures with private keys. Future versions may integrate GPG or X.509 signing.
- The provenance model is pipeline-centric rather than PROV-compliant; future work will generate machine-readable PROV-O [Moreau and Missier, 2013] records alongside the embedded watermarks.

8.6 Relationship to Software Supply Chain Integrity

The steganographic provenance layer operates at the *document level*—it certifies the integrity of a specific PDF artifact. A complementary concern is *build-level* provenance: certifying that the pipeline itself was executed with verified source code and dependencies. Frameworks such as in-toto [Torres-Arias et al., 2019] and SLSA (Supply-chain Levels for Software Artifacts) address this concern by defining attestation chains from source commit through build steps to final artifact. Future versions of the Docxology Template may generate SLSA-compatible provenance attestations alongside the steganographic watermarks, creating a two-layer provenance model: in-toto attests that the build pipeline was executed with the claimed source code, while the steganographic layer attests that the PDF was produced by that pipeline at a specific time.

8.7 Relationship to FAIR and Formal Provenance Standards

The steganographic layer supports the FAIR for Research Software (FAIR4RS) principles [Barker et al., 2022] at the artifact level. PDFs carry embedded metadata (Findability) in standardized XMP format (Interoperability). The SHA-256 hash manifest enables persistent integrity verification (a prerequisite for Reusability). The Documentation Duality standard ensures that the software producing the artifact is inspectable and well-documented (satisfying FAIRsoft [Garijo et al., 2024] metadata and documentation indicators).

Full PROV-compliant provenance traces—capturing the derivation chain from source data through analysis scripts to rendered PDF—are a natural extension and a primary target for future development.

9 Appendices

9.1 Appendix A: Pipeline Stage Reference

Stage	Script	Input	Output	Failure Mode
00	`00_setup_environment.py`	System environment	Validated env, directories	Hard fail
01	`01_run_tests.py`	`tests/`, `projects/*/tests/`	Coverage JSON, test reports	Configurable
02	`02_run_analysis.py`	`projects/*/scripts/*.py`	Figures, data files	Hard fail
03	`03_render_pdf.py`	`manuscript/*.md`, `config.yaml`	PDF in `output/`	Hard fail
04	`04_validate_output.py`	`output/` contents	Validation report	Warning
05	`05_copy_outputs.py`	`output/` artifacts	Organized copies	Soft fail
06	`06_llm_review.py`	Rendered manuscript	Executive summary, reviews	Skippable
07	`07_generate_execution_report.py`	All stage outputs	JSON + Markdown report	Soft fail

9.2 Appendix B: Configuration Reference (`config.yaml`)

```yaml
paper:
  title: "Paper Title"
  subtitle: "Optional Subtitle"
  version: "1.0"
  date: "2026-03-08"

authors:
  - name: "Author Name"
    orcid: "0000-0000-0000-0000"
    email: "author@example.com"
    affiliation: "Institution"
    corresponding: true

publication:
  doi: "10.5281/zenodo.XXXXXX"
  journal: "Target Journal"
  volume: "1"
  pages: "1-10"
  year: "2026"

keywords:
  - "keyword1"
  - "keyword2"

metadata:
  license: "Apache License 2.0"
  language: "en"
```

```yaml
llm:
  reviews:
    enabled: true
    types: [executive_summary, quality_review]
  translations:
    enabled: false

testing:
  max_test_failures: 0
  max_infra_test_failures: 3
  max_project_test_failures: 0
```

9.3 Appendix C: Repository Directory Structure

```
docxology/template/
   infrastructure/           # Layer 1: Shared services (137 .py files)
       core/                 # 28 files - logging, config, exceptions
       documentation/        # 6 files - figure management, glossary
       llm/                  # 30 files - Ollama integration, literature
       project/              # 2 files - project discovery
       publishing/           # 9 files - citation generation, Zenodo
       rendering/            # 12 files - Pandoc + XeLaTeX + reports
       reporting/            # 14 files - coverage parsing, reports
       scientific/           # 6 files - stability, benchmarking
       steganography/        # 8 files - watermarking, hashing
       validation/           # 22 files - PDF + Markdown validation
   scripts/                  # Pipeline orchestration
       00_setup_environment.py
       01_run_tests.py
       02_run_analysis.py
       03_render_pdf.py
       04_validate_output.py
       05_copy_outputs.py
       06_llm_review.py
       07_generate_executive_report.py
       execute_pipeline.py
   projects/                 # Layer 2: Research workspaces
       code_project/         # Gradient descent exemplar
       cognitive_case_diagrams/  # Category theory + linguistics
       template/             # This self-referential project
   tests/                    # Infrastructure test suite (148 files, 3,053 tests)
   docs/                     # Repository documentation (90+ files, 12 subdirectories)
   run.sh                    # Interactive TUI orchestrator
   secure_run.sh             # Steganographic pipeline wrapper
   AGENTS.md                 # System-level AI agent documentation
   CLAUDE.md                 # Global AI coding assistant instructions
   README.md                 # Human-readable project overview
   pyproject.toml            # Root project configuration (uv)
```

9.4 Appendix D: Exemplar Project Summary

Project	Domain	src/ Modules	Test Count	Coverage	Figures	Pages
`code_project`	Numerical optimization	`optimizer.py` (248 lines)	58	96.6%	6	~20
`cognitive_case_study`	Category theory / linguistics	12 modules, 17 DisCoPy renderers	257	~96%	25+	~77
`template`	Meta-architecture	`introspection.py`	36	90%+	3	~5

9.5 Appendix E: Documentation Inventory

The repository maintains documentation at three levels:

Level	Files	Purpose
Repository root	`AGENTS.md`, `CLAUDE.md`, `README.md`, `RUN_GUIDE.md`	Global navigation and AI agent context
`docs/` directory	90+ files across 12 subdirectories	User guides, API reference, troubleshooting
Per-directory	`AGENTS.md` + `README.md` at every directory	Documentation Duality standard

The `docs/` subdirectories cover: `core/` (essential docs), `guides/` (skill levels 1–12), `architecture/` (system design), `usage/` (content authoring), `operational/` (build, config, logging, troubleshooting), `reference/` (API, FAQ, glossary), `modules/` (10 infrastructure modules), `development/` (contributing, testing), `best-practices/` (version control, migration), `prompts/` (9 AI prompt templates), `security/` (steganography, hashing), and `audit/` (review reports).

9.6 Appendix F: Comparative Tool Matrix

Capability	Docxology	Snakemake	Nextflow	CWL	Quarto	Jupyter Book	R Markdown
Pipeline orchestration					—	—	—
Manuscript rendering		—	—	—			
Code execution							
Testing enforcement		—	—	—	—	—	—
Coverage thresholds		—	—	—	—	—	—
Cryptographic provenance		—	—	—	—	—	—
Steganographic watermarking		—	—	—	—	—	—
Multi-project management		—	—	—	—	—	—

Capability	Docxology	Snakemake	Nextflow	CWL	Quarto	Jupyter Book	R Markdown
AI-agent documentation		—	—	—	—	—	—
Interactive TUI		—	—	—	—	—	—
Container support	—				—	—	—
Distributed execution	—				—	—	—
DAG parallelism	—				—	—	—
Multi-language (R/Julia)	—		—				

References

J. J. Allaire, Charles Teague, Carlos Scheidegger, Yihui Xie, and Christophe Dervieux. Quarto, 2024. URL https://github.com/quarto-dev/quarto-cli.

Peter Amstutz, Michael R. Crusoe, Nebojša Tijanić, Brad Chapman, John Chilton, Michael Heber, Andrey Kartashov, Dan Leehr, Hervé Ménager, Maya Nedeljkovich, Matt Scales, Stian Soiland-Reyes, and Luka Stojanovic. Common workflow language, v1.0. *Specification, Common Workflow Language working group*, 2016. doi: 10.6084/m9.figshare.3115156.v2.

Monya Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533(7604):452–454, 2016. doi: 10.1038/533452a.

Lorena A. Barba. Terminologies for reproducible research. *arXiv preprint arXiv:1802.03311*, 2018. doi: 10.48550/arXiv.1802.03311.

Michelle Barker, Neil P. Chue Hong, Daniel S. Katz, Anna-Lena Lamprecht, Carlos Martinez-Ortiz, Fotis Psomopoulos, Jennifer Harrow, Leyla Jael Castro, Morane Gruenpeter, Paula Andrea Martinez, and Tom Honeyman. Introducing the FAIR principles for research software. *Scientific Data*, 9(1):622, 2022. doi: 10.1038/s41597-022-01710-x.

Carl Boettiger. An introduction to docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1):71–79, 2015. doi: 10.1145/2723872.2723882.

Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017. doi: 10.1038/nbt.3820.

Leonard P. Freedman, Iain M. Cockburn, and Timothy S. Simcoe. The economics of reproducibility in preclinical research. *PLOS Biology*, 13(6):e1002165, 2015. doi: 10.1371/journal.pbio.1002165.

Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995. ISBN 0-201-63361-2.

Daniel Garijo, Luis García, Matthias Barker, Rubén Chaves, Oscar Corcho, et al. FAIRsoft—a practical implementation of FAIR principles for research software. *Bioinformatics*, 40(8):btae464, 2024. doi: 10.1093/bioinformatics/btae464.

Robert Gentleman and Duncan Temple Lang. Statistical analyses and reproducible research. *Journal of Computational and Graphical Statistics*, 16(1):1–23, 2007. doi: 10.1198/106186007X178663.

Morane Gruenpeter, Daniel S. Katz, Anna-Lena Lamprecht, Tom Honeyman, Daniel Garijo, Alexander Struck, Anna Niehues, Paula Andrea Martinez, Leyla Jael Castro, Tovo Rabemanantsoa, Neil P. Chue Hong, Carlos Martinez-Ortiz, Laurents Sesink, Matthias Liffers, et al. Defining research software: A controversial discussion. In *SSRN Preprint*, 2021. doi: 10.2139/ssrn.3884311.

Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks — a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90, 2016. doi: 10.3233/978-1-61499-649-1-87.

Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984. doi: 10.1093/comjnl/27.2.97.

Johannes Köster and Sven Rahmann. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012. doi: 10.1093/bioinformatics/bts480.

Sam Lau and Philip J. Guo. The design space of LLM-based AI coding assistants: An analysis of 90 systems. In *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2025. URL https://pg.ucsd.edu/publications/ai-coding-assistants-design-space_VLHCC-2025.pdf.

Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. ISBN 978-0132350884.

Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium, 2013. URL https://www.w3.org/TR/prov-dm/.

Daniel Nüst, Markus Konkol, Edzer Pebesma, Christian Kray, Marc Schutzeichel, Holger Przibytzin, and Jörg Lorenz. Opening the publication process with executable research compendia. *D-Lib Magazine*, 23 (1/2), 2017. doi: 10.1045/january2017-nuest.

Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.

Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.

Eric Schulte, Dan Davison, Thomas Dye, and Carsten Dominik. A multi-language computing environment for literate programming and reproducible research. *Journal of Statistical Software*, 46(3):1–24, 2012. doi: 10.18637/jss.v046.i03.

Victoria Stodden, Marcia McNutt, David H. Bailey, Ewa Deelman, Yolanda Gil, Brooks Hanson, Michael A. Heroux, John P. A. Ioannidis, and Michela Taufer. Enhancing reproducibility for computational methods. *Science*, 354(6317):1240–1241, 2016. doi: 10.1126/science.aah6168.

Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pages 1393–1410, 2019. URL https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E. Bourne, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi: 10.1038/sdata.2016.18.

Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6):e1005510, 2017. doi: 10.1371/journal.pcbi.1005510.

Yihui Xie, J. J. Allaire, and Garrett Grolemund. *R Markdown: The Definitive Guide*. Chapman and Hall/CRC, 2018. ISBN 978-1138359338. doi: 10.1201/9781138359444.